

Messages > Building with Claude

Stop reasons and fallback

Learn what each `stop_reason` value means and how to handle truncation, tool use, paused turns, and refusals in your application.

 Copy page 

Every Messages API response includes a `stop_reason` field that tells you why Claude stopped generating. Check this field to decide whether to use the response as-is, continue the conversation, retry, or fall back to another model.

For the full response schema, see the [Messages API reference](#).

Quick reference

Value	When it occurs	What to do
<code>end_turn</code>	Claude finished its response naturally.	Use the response.
<code>max_tokens</code>	The response reached your <code>max_tokens</code> limit.	Raise <code>max_tokens</code> or continue the response .
<code>stop_sequence</code>	Claude emitted one of your <code>stop_sequences</code> .	Read <code>stop_sequence</code> to see which one fired.
<code>tool_use</code>	Claude is calling a tool.	Run the tool and return the result. A server tool call still missing its result block completes in a later response.
<code>pause_turn</code>	A server-tool loop reached its iteration limit.	Send the assistant content back to continue.
<code>refusal</code>	Claude declined to respond.	Read <code>stop_details</code> and retry on a fallback model .

`window_exceede
d`

model's context
window.

Treat the response as truncated.

The `stop_reason` field

The `stop_reason` field is part of every successful Messages API response. Unlike errors, which indicate failures in processing your request, `stop_reason` tells you why Claude completed its response generation.

Example response



```
{
  "id": "msg_01234",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "Here's the answer to your question..."
    }
  ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "stop_details": null,
  "usage": {
    "input_tokens": 100,
    "output_tokens": 50
  }
}
```

Stop reason values

`end_turn`

The most common stop reason. Indicates Claude finished its response naturally.

cURL

CLI

Python

TypeScript

C#

Go

Java

PHP

Ruby



```
response = client.messages.create(  
    model="claude-opus-5",  
    max_tokens=1024,  
    messages=[{"role": "user", "content": "Hello!"}],  
)  
if response.stop_reason == "end_turn":  
    # Process the complete response  
    for block in response.content:  
        if block.type == "text":  
            print(block.text)
```

Empty responses with end_turn



max_tokens

Claude stopped because it reached the `max_tokens` limit specified in your request.

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
client = anthropic.Anthropic()  
# Request with limited tokens  
response = client.messages.create(  
    model="claude-opus-5",  
    max_tokens=10,  
    messages=[{"role": "user", "content": "Explain quantum physics"}],  
)  
  
if response.stop_reason == "max_tokens":  
    # Response was truncated  
    print("Response was cut off at token limit")  
    # Consider making another request to continue
```

Incomplete tool use blocks



stop_sequence

Claude encountered one of your custom stop sequences.

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
model="claude-opus-5",
max_tokens=1024,
stop_sequences=["END", "STOP"],
messages=[{"role": "user", "content": "Generate text until you say END"}],
)

if response.stop_reason == "stop_sequence":
    print(f"Stopped at sequence: {response.stop_sequence}")
```

tool_use

Claude is calling a tool and expects you to run it.

 For most tool use implementations, use the [tool runner](#), which automatically handles tool execution, result formatting, and conversation management.

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
"name": "get_weather",
"description": "Get the current weather in a given location",
"input_schema": {
    "type": "object",
    "properties": {
        "location": {"type": "string", "description": "City and state"},
    },
    "required": ["location"],
},
},
}

def execute_tool(name, tool_input):
    """Execute a tool and return the result."""
    return f"Weather in {tool_input.get('location', 'unknown')}: 72°F"

response = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=[weather_tool],
    messages=[{"role": "user", "content": "What is the weather in San Francisco?"}],
)

if response.stop_reason == "tool_use":
    # Extract and execute the tool
    for block in response.content:
        if block.type == "tool_use":
            result = execute_tool(block.name, block.input)
            # Return result to Claude for final response
```

A `tool_use` response can also contain a `server_tool_use` block whose `id` has no matching result block. That server tool call is not finished, and this response does not carry its result. In the common case, Claude calls a server tool and one of your client tools in the same group of parallel tool calls: the API returns without running the server tool so that you can run the client tools first. There is no other marker for the state; detect it by checking each `server_tool_use` or `mcp_tool_use` block's `id` for a matching result block.

rather than from Claude directly, and its `caller` field names the `code_execution` block that called it. That code has already started: it is paused waiting for your `tool_result` blocks, and sending them resumes the execution instead of starting a deferred tool. The `code_execution` block's own result block arrives once the code finishes, which can take more than one round of tool results. The follow-up user message itself is the same in both cases; with programmatic tool calling, also pass back the `id` from the response's `container` field, as that page shows.

A mixed `tool_use` response



```
{
  "stop_reason": "tool_use",
  "content": [
    {
      "type": "server_tool_use",
      "id": "srvtoolu_01HxbWnMRmbWyMfUtJKC45rA",
      "name": "web_search",
      "input": { "query": "example article" }
    },
    {
      "type": "tool_use",
      "id": "toolu_01PjgRJLbXrXEMZwDNYLnBqk",
      "name": "run_command",
      "input": { "command": "uname -a" }
    }
  ]
}
```

The continuation is a user message of `tool_result` blocks, one for every `tool_use` block in the response (see [Handle tool calls](#)), with two extra rules: that message must contain nothing except the `tool_result` blocks, and the request must keep the same `tools` array. A resume request that no longer defines the waiting server tool fails with a 400 whose message ends `but no `web_search` tool was provided`. The API attaches your results to the still-open assistant turn, runs the deferred server tool (for paused code execution, resumes it), and continues the turn. For a server tool Claude called directly, the next response's `content` starts with the result block that answers the previous response's `server_tool_use` `id`.

The follow-up user message



```
"content": [  
  {  
    "type": "tool_result",  
    "tool_use_id": "toolu_01PjgRJLbXrXEMZwDNYLnBqk",  
    "content": "Linux demo-host 6.8.0-52-generic x86_64 GNU/Linux"  
  }  
]  
}
```

Adding anything after the `tool_result` blocks in that user message, such as text, ends the assistant turn; for a server tool Claude called directly, the request then fails with a 400 `invalid_request_error` that names the unresolved server tool:

```
`web_search` tool use with id `srvtoolu_01HxbWnMRmbWyMfUtJKC45rA` was found  
without a corresponding `web_search_tool_result` block
```

Leaving out a `tool_result`, or putting one after other content, fails earlier with the standard `tool_use` ids were found without `tool_result` blocks immediately after error instead. To give Claude more input, send it as a separate user message after the turn completes.

pause_turn

Returned when the server-side sampling loop reaches its iteration limit while executing server tools such as web search. The default limit is 10 iterations per request.

When this happens, the response may contain a `server_tool_use` block without a corresponding result block. To let Claude finish processing, continue the conversation by sending the response back as-is. A response that leaves a client `tool_use` block waiting on you never has a `stop_reason` of `pause_turn`: when Claude stops to call your tools, `stop_reason` is `tool_use`, and you continue it by sending the client `tool_result` blocks instead of the response itself.

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
max_tokens=4096,
tools=[{"type": "web_search_20250305", "name": "web_search"}],
messages=[{"role": "user", "content": "Search for latest AI news"}],
)

if response.stop_reason == "pause_turn":
    # Continue the conversation by sending the response back
    messages = [
        {"role": "user", "content": "Search for latest AI news"},
        {"role": "assistant", "content": response.content},
    ]
    continuation = client.messages.create(
        model="claude-opus-5",
        max_tokens=4096,
        messages=messages,
        tools=[{"type": "web_search_20250305", "name": "web_search"}],
    )
```

📝 Your application should handle `pause_turn` in any agent loop that uses server tools. Add the assistant's response to your messages array and make another API request to let Claude continue.

refusal

Claude declined to generate a response. Safety classifiers return this stop reason as a normal HTTP 200 response, not an error.

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "[Unsafe request]"}],
)

if response.stop_reason == "refusal":
    # Claude declined to respond
    print("Claude was unable to process this request")
    # Consider rephrasing or modifying the request
```

updating your API calls to use Haiku 4.5 (`claude-haiku-4-5-20251001`), which has different usage restrictions. Learn more about [understanding Sonnet 4.5's API safety filters](#).

On a refusal, the `stop_details` object identifies the policy category that triggered it. The categories and the full refusal response shape are covered on [Refusals and fallback](#).

`stop_details` is `null` for all stop reasons other than `refusal`.

A refused request on Claude Fable 5 or Claude Opus 5 can usually be served by retrying on another Claude model, and [Refusals and fallback](#) shows how to set up that retry, server-side or in your client. [Fallback credit](#) covers how to avoid paying the prompt-cache cost twice when you build the retry yourself.

model_context_window_exceeded

Claude stopped because it reached the model's context window limit. This lets you request the maximum possible tokens without knowing the exact input size.

 This stop reason is currently typed only in the SDKs' `beta` namespace, so the following examples call `client.beta.messages` and use the `Beta`-prefixed types. On Sonnet 4.5 and newer models the API returns this value without a beta header. For earlier models, add the `model-context-window-exceeded-2025-08-26` beta header to enable it.

cURL CLI **Python** TypeScript C# Go Java PHP Ruby 

```
# Request with maximum tokens to get as much as possible
response = client.beta.messages.create(
    model="claude-opus-5",
    max_tokens=20000, # Python SDK requires streaming for max_tokens above
~21k
    messages=[
        {"role": "user", "content": "Large input that uses most of context
window..."}
    ],
)

if response.stop_reason == "model_context_window_exceeded":
    # Response hit context window limit before max_tokens
    print("Response reached model's context window limit")
    # The response is still valid but was limited by context window
```

Always check stop_reason

Make it a habit to check the `stop_reason` in your response handling logic:

Python TypeScript C# Go Java PHP Ruby



```
def handle_response(response):
    if response.stop_reason == "tool_use":
        return handle_tool_use(response)
    elif response.stop_reason == "max_tokens":
        return handle_truncation(response)
    elif response.stop_reason == "model_context_window_exceeded":
        return handle_context_limit(response)
    elif response.stop_reason == "pause_turn":
        return handle_pause(response)
    elif response.stop_reason == "refusal":
        return handle_refusal(response)
    else:
        # Handle end_turn and other cases
        return next(
            (block.text for block in response.content if block.type == "text"),
            ""
        )
```

Handle truncated responses gracefully

When a response is truncated because of token limits or the context window, append a notice so the reader knows the output is incomplete. To continue generating from where the response left off instead, see [Ensuring complete responses](#).

Python TypeScript C# Go Java PHP Ruby



```
def handle_truncated_response(response):
    text = next((block.text for block in response.content if block.type ==
    "text"), "")
    if response.stop_reason in ["max_tokens", "model_context_window_exceeded"]:
        if response.stop_reason == "max_tokens":
            note = "[Response truncated due to max_tokens limit]"
        else:
            note = "[Response truncated due to context window limit]"
        return f"{text}\n\n{note}"
    return text
```

When using `server tools`, the API may return `pause_turn` if the server-side sampling loop reaches its iteration limit (default 10). Handle this by continuing the conversation:

Python

TypeScript

C#

Go

Java

PHP

Ruby



```
def handle_server_tool_conversation(client, user_query, tools,
max_continuations=5):
    """
    Handle server tool conversations that may require multiple continuations.

    The server runs a sampling loop when executing server tools. If the loop
    reaches its iteration limit, the API returns pause_turn. Continue the
    conversation by sending the response back to let Claude finish.
    """
    messages = [{"role": "user", "content": user_query}]

    for _ in range(max_continuations):
        response = client.messages.create(
            model="claude-opus-5", max_tokens=4096, messages=messages,
tools=tools
        )

        if response.stop_reason != "pause_turn":
            # Claude finished processing - return the final response
            return response

        # pause_turn: replace the full message list to maintain alternating
roles
        messages = [
            {"role": "user", "content": user_query},
            {"role": "assistant", "content": response.content},
        ]

    # Reached max continuations - return the last response
    return response
```

Stop reasons vs. errors

It's important to distinguish between `stop_reason` values and actual errors:

Stop reasons (successful responses)

- Part of the response body
- Indicate why generation stopped normally

Errors (failed requests)

- HTTP status codes 4xx or 5xx
- Indicate request processing failures
- Response contains error details

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
client = anthropic.Anthropic()

try:
    response = client.messages.create(
        model="claude-opus-5",
        max_tokens=1024,
        messages=[{"role": "user", "content": "Hello!"}],
    )

    # Handle successful response with stop_reason
    if response.stop_reason == "max_tokens":
        print("Response was truncated")

except anthropic.APIStatusError as e:
    # Handle actual errors
    if e.status_code == 429:
        print("Rate limit exceeded")
    elif e.status_code == 500:
        print("Server error")
```

Streaming considerations

When using streaming, `stop_reason` is:

- `null` in the initial `message_start` event
- Provided in the `message_delta` event
- Not provided in any other events

cURL CLI **Python** TypeScript C# Go Java PHP Ruby



```
with client.messages.stream(  
    model="claude-opus-5",  
    max_tokens=1024,  
    messages=[{"role": "user", "content": "Hello!"}],  
) as stream:  
    for event in stream:  
        if event.type == "message_delta":  
            stop_reason = event.delta.stop_reason  
            if stop_reason:  
                print(f"Stream ended with: {stop_reason}")
```

Common patterns

Handling tool use workflows

💡 **Simpler with tool runner:** The following example shows manual tool handling. For most use cases, the tool runner automatically handles tool execution with much less code.

Python TypeScript C# Go Java PHP Ruby



```
def complete_tool_workflow(client, user_query, tools):  
    messages = [{"role": "user", "content": user_query}]  
  
    while True:  
        response = client.messages.create(  
            model="claude-opus-5", max_tokens=1024, messages=messages,  
            tools=tools  
        )  
  
        if response.stop_reason == "tool_use":  
            # Execute tools and continue  
            tool_results = execute_tools(response.content)  
            messages.append({"role": "assistant", "content": response.content})  
            messages.append({"role": "user", "content": tool_results})  
        else:  
            # Final response  
            return response
```

Python TypeScript C# Go Java PHP Ruby



```
def get_complete_response(client, prompt, max_attempts=3):
    messages = [{"role": "user", "content": prompt}]
    full_response = ""

    for _ in range(max_attempts):
        response = client.messages.create(
            model="claude-opus-5", messages=messages, max_tokens=4096
        )

        full_response += next(
            (block.text for block in response.content if block.type == "text"),
            ""
        )

        if response.stop_reason != "max_tokens":
            break

    # Continue from where it left off
    messages = [
        {"role": "user", "content": prompt},
        {"role": "assistant", "content": full_response},
        {"role": "user", "content": "Please continue from where you left
off."},
    ]

    return full_response
```

Getting maximum tokens without knowing input size

With the `model_context_window_exceeded` stop reason, you can request the maximum possible tokens without calculating input size:

Python TypeScript C# Go Java PHP Ruby



```
Get as many tokens as possible within the model's context window
without needing to calculate input token count
"""
response = client.beta.messages.create(
    model="claude-opus-5",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=20000, # Python SDK requires streaming for max_tokens above
~21k
)

if response.stop_reason == "model_context_window_exceeded":
    # Got the maximum possible tokens given input size
    print(
        f"Generated {response.usage.output_tokens} tokens (context limit
reached)"
    )
elif response.stop_reason == "max_tokens":
    # Got exactly the requested tokens
    print(f"Generated {response.usage.output_tokens} tokens (max_tokens
reached)")
else:
    # Natural completion
    print(f"Generated {response.usage.output_tokens} tokens (natural
completion)")

    return next((block.text for block in response.content if block.type ==
"text"), "")
```

Next steps



Refusals and fallback

Retry refused requests on a fallback model, server-side or in your client.



Tool Runner (SDK)

Let the SDK manage the `tool_use` loop, result formatting, and retries for you.

Read `stop_reason` from the `message_delta` event when streaming.



Errors

Handle 4xx and 5xx HTTP errors, which are distinct from stop reasons.

Cette page vous a-t-elle été utile ?



Claude Platform Docs



Solutions

AI agents

Code modernization

Coding

Customer support

Financial services

Government

Higher education

K-12 teachers

Life sciences

Partners

Claude on AWS

Claude on Google Cloud

Learn

Blog

Courses

Use cases

Connectors

Customer stories

Engineering at Anthropic

Events

Powered by Claude

Service partners

Startups program

Company

Anthropic

Research

News

Responsible Scaling Policy

Security and compliance

Transparency

Help and security

Availability

Status

Support

Discord

Terms and policies

Privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy